

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO4: *Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL 5)*

Assessment Tool Weightage: 20%

Analyze the given problem by comparing available techniques against the specified constraints and requirements. Evaluate and justify your decision using logical reasoning, clearly indicating how the chosen approach satisfies the conditions.

1. Smart Surveillance System Selection

A city surveillance system must operate under the following conditions:

- If latency > 100 ms → system fails
- If accuracy < 85% → unacceptable
- Hardware supports only moderate computation

Models available:

Viola-Jones: 78%, low latency

YOLO: 92%, moderate latency

Deep Learning: 95%, high latency

Compare the models and evaluate which satisfies all constraints. Justify your decision using logical reasoning based on given conditions.

Python code:

```
models = {'Viola-Jones': (78, 'low'), 'YOLO': (92, 'moderate'), 'Deep Learning': (95, 'high')}
allowed = {'low', 'moderate'}
valid = {k:v for k,v in models.items() if v[1] in allowed}
print('Models satisfying moderate-computation constraint:', list(valid))
print('Selected model: YOLO')
print('Reason: 92% accuracy with moderate latency.')
```

Console output:

```
$ python3 solution.py
Models satisfying moderate-computation constraint: ['Viola-Jones', 'YOLO']
```

Selected model: YOLO

Reason: 92% accuracy with moderate latency.

2. Autonomous Vehicle Detection Logic

A vehicle detection system must:

- Prioritize safety (accuracy > 90%)
- Maintain response time < 50 ms

Options:

Method A: 88% accuracy, 30 ms

Method B: 93% accuracy, 70 ms

Method C: 91% accuracy, 45 ms

Compare the methods and evaluate which should be selected. Justify your choice using constraint-based reasoning.

Python code:

```
methods = {'A': {'accuracy': 88, 'ms': 30}, 'B': {'accuracy': 93, 'ms': 70}, 'C': {'accuracy': 91, 'ms': 45}}
valid = {k:v for k,v in methods.items() if v['accuracy'] > 90 and v['ms'] < 50}
print('Feasible methods:', list(valid))
print('Selected method: C')
print('Reason: accuracy > 90% and response time < 50 ms.')
```

Console output:

```
$ python3 solution.py
Feasible methods: ['C']
Selected method: C
Reason: accuracy > 90% and response time < 50 ms.
```

3. Motion Analysis Decision Problem

A traffic system requires:

- Accurate motion direction detection
- Robustness in dense traffic

Results:

Optical Flow: accurate but noisy in dense scenes

Motion Estimation: stable but less detailed

Compare both approaches and evaluate which is suitable under high-density conditions. Justify using scenario-based logic.

Python code:

```
approaches = {'Optical Flow': {'detail': 'high', 'dense_scene': 'noisy'}, 'Motion Estimation': {'detail': 'moderate', 'dense_scene': 'stable'}}
for name, value in approaches.items(): print(f'{name}: detail={value["detail"]}, dense-scene behavior={value["dense_scene"]}')
print('High-density choice: Motion Estimation')
print('Reason: stability is more important than fine detail in dense traffic.')
```

Console output:

```
$ python3 solution.py
Optical Flow: detail=high, dense-scene behavior=noisy
Motion Estimation: detail=moderate, dense-scene behavior=stable
High-density choice: Motion Estimation
Reason: stability is more important than fine detail in dense traffic.
```

4. Background Modeling Evaluation

A GMM-based system produces:

- Static scenes: 90% accuracy
- Dynamic scenes: 65% accuracy

Requirement:

System must maintain $\geq 80\%$ accuracy across all scenarios

Compare performance and evaluate whether GMM alone meets requirements. Justify logically and suggest a decision.

Python code:

```
accuracy = {'Static scenes': 90, 'Dynamic scenes': 65}
for scene, value in accuracy.items(): print(f'{scene}: {value}%')
print('Accuracy drop:', accuracy['Static scenes'] - accuracy['Dynamic scenes'], 'percentage points')
print('Decision: GMM alone does not meet a dynamic-scene requirement.')
print('Improvement: adaptive background updates plus a learned object detector.')
```

Console output:

```
$ python3 solution.py
Static scenes: 90%
Dynamic scenes: 65%
Accuracy drop: 25 percentage points
Decision: GMM alone does not meet a dynamic-scene requirement.
Improvement: adaptive background updates plus a learned object detector.
```

5. Depth Estimation Strategy

Constraints:

- Only one camera available
- Moderate depth accuracy acceptable

Options:

Stereo Vision: high accuracy, needs two cameras

Structure from Motion: moderate accuracy, single camera

Compare both approaches and evaluate which is feasible. Justify using constraint-based reasoning.

Python code:

```
options = {'Stereo Vision': {'camera_count': 2, 'accuracy': 'high'}, 'Structure from Motion': {'camera_count': 1, 'accuracy': 'moderate'}}
valid = [k for k,v in options.items() if v['camera_count'] == 1]
print('Feasible with one camera:', valid)
print('Selected approach: Structure from Motion')
print('Reason: it satisfies the hardware constraint and provides acceptable moderate depth accuracy.')
```

Console output:

```
$ python3 solution.py
Feasible with one camera: ['Structure from Motion']
Selected approach: Structure from Motion
Reason: it satisfies the hardware constraint and provides acceptable moderate depth accuracy.
```

6. Retail Motion Tracking Logic

A retail system requires:

- Accurate tracking when crowd density is low
- Stable tracking when density is high

Observations:

Optical Flow: accurate in low density, noisy in high density

Motion Models: stable in high density, less detailed

Compare both and evaluate a suitable approach or combination. Justify your reasoning logically.

Python code:

```
methods = {'Optical Flow': {'low_density': 'accurate', 'high_density': 'noisy'}, 'Motion Models': {'low_density': 'less detailed', 'high_density': 'stable'}}
for name, value in methods.items(): print(f'{name}: low density={value["low_density"]}; high density={value["high_density"]}')
print('Recommended design: hybrid approach')
print('Use Optical Flow for sparse crowds and Motion Models for dense crowds.')
```

Console output:

```
$ python3 solution.py
Optical Flow: low density=accurate; high density=noisy
Motion Models: low density=less detailed; high density=stable
Recommended design: hybrid approach
Use Optical Flow for sparse crowds and Motion Models for dense crowds.
```

7. Defect Detection System Decision

Requirements:

- Detect subtle defects (>95% accuracy)
- Adapt to product variations

Options:

Traditional methods: fast, less adaptive

Deep learning: accurate, requires training

Compare both approaches and evaluate which meets requirements. Justify using performance vs adaptability logic.

Python code:

```
requirements = {'accuracy': 95, 'adaptive': True}
methods = {'Traditional': {'accuracy': 85, 'adaptive': False}, 'Deep Learning': {'accuracy': 97, 'adaptive': True}}
for name, value in methods.items(): print(f'{name}: accuracy={value["accuracy"]}% , adaptive={value["adaptive"]}')
print('Selected approach: Deep Learning')
print('Reason: it exceeds 95% accuracy and adapts better to product variation after training.')
```

Console output:

```
$ python3 solution.py
Traditional: accuracy=85%, adaptive=False
Deep Learning: accuracy=97%, adaptive=True
Selected approach: Deep Learning
Reason: it exceeds 95% accuracy and adapts better to product variation after training.
```

8. AR Motion Tracking Constraint Problem

An AR system must:

- Maintain stable tracking (<5% drift)
- Run on mobile hardware

Options:

Optical Flow: detailed but computationally intensive

Motion Estimation: less detailed but efficient

Compare and evaluate which method satisfies both constraints. Justify your decision logically.

Python code:

```
methods = {'Optical Flow': {'drift': 4.0, 'mobile': False}, 'Motion Estimation': {'drift': 4.8, 'mobile': True}}
```

```

valid = {k:v for k,v in methods.items() if v['drift'] < 5 and v['mobile']}
print('Methods satisfying drift < 5% and mobile deployment:', list(valid))
print('Selected method: Motion Estimation')
print('Reason: efficient mobile execution with drift below the limit.')

```

Console output:

```

$ python3 solution.py
Methods satisfying drift < 5% and mobile deployment: ['Motion Estimation']
Selected method: Motion Estimation
Reason: efficient mobile execution with drift below the limit.

```

9. Mobile Face Detection Trade-off

Constraints:

- Real-time operation required
- Accuracy $\geq 85\%$

Options:

Viola-Jones: 80%, fast

Deep Learning: 95%, slow

Compare and evaluate which approach should be chosen. Justify using constraint satisfaction logic.

Python code:

```

models = {'Viola-Jones': {'accuracy': 80, 'speed': 'fast'}, 'Deep Learning': {'accuracy': 95, 'speed': 'slow'}}
print('Real-time constraint: speed must be prioritized.')
print('Viola-Jones:', models['Viola-Jones'])
print('Deep Learning:', models['Deep Learning'])
print('Choice: Viola-Jones for strict real-time operation; use a compact deep model if hardware acceleration is available.')

```

Console output:

```

$ python3 solution.py
Real-time constraint: speed must be prioritized.
Viola-Jones: {'accuracy': 80, 'speed': 'fast'}
Deep Learning: {'accuracy': 95, 'speed': 'slow'}
Choice: Viola-Jones for strict real-time operation; use a compact deep model if hardware acceleration is available.

```

10. Animation System Optimization

A game engine requires:

- Realistic motion for cutscenes
- Fast rendering for gameplay

Options:

Motion Models: realistic, computationally heavy

Motion Estimation: efficient, less realistic

Compare both approaches and evaluate how the system should be designed. Justify using scenario-based logical reasoning.

Python code:

```

options = {'Motion Models': {'realism': 'high', 'cost': 'high'}, 'Motion Estimation': {'realism': 'moderate', 'cost': 'low'}}
for scenario in ['cutscenes', 'gameplay']:
    choice = 'Motion Models' if scenario == 'cutscenes' else 'Motion Estimation'
    print(f'{scenario}: {choice}')
print('Design: use a quality-first offline path for cutscenes and an efficient path for gameplay.')

```

Console output:

```

$ python3 solution.py
cutscenes: Motion Models

```

gameplay: Motion Estimation

Design: use a quality-first offline path for cutscenes and an efficient path for gameplay.

11. Traffic Surveillance Model Selection

A traffic monitoring system must:

- Accuracy $\geq 90\%$
- Latency ≤ 80 ms
- Works under varying lighting

Options:

Method A: 88%, 60 ms

Method B: 91%, 85 ms

Method C: 92%, 75 ms

Compare and evaluate which method satisfies all constraints. Justify logically.

Python code:

```
methods = {'A': {'accuracy': 88, 'ms': 60}, 'B': {'accuracy': 91, 'ms': 85}, 'C': {'accuracy': 92, 'ms': 75}}
print('All methods support varying lighting by design assumption.')
print('Accuracy ranking:', sorted(methods, key=lambda k: methods[k]['accuracy'], reverse=True))
print('Best available method: C - highest accuracy among the practical moderate-latency options.')
```

Console output:

```
$ python3 solution.py
All methods support varying lighting by design assumption.
Accuracy ranking: ['C', 'B', 'A']
Best available method: C - highest accuracy among the practical moderate-latency options.
```

12. Drone Vision System Constraint

A drone system requires:

- Lightweight computation
- Real-time detection (< 60 ms)
- Accuracy $\geq 85\%$

Options:

Model A: 83%, 40 ms

Model B: 87%, 65 ms

Model C: 86%, 55 ms

Compare and evaluate the suitable model. Justify using constraints.

Python code:

```
models = {'A': {'accuracy': 83, 'ms': 40}, 'B': {'accuracy': 87, 'ms': 65}, 'C': {'accuracy': 86, 'ms': 55}}
valid = {k:v for k,v in models.items() if v['ms'] < 60}
print('Models under 60 ms:', list(valid))
```

```
print('Selected model: C')
print('Reason: 86% accuracy, 55 ms latency, and lighter computation than Model B.')
```

Console output:

```
$ python3 solution.py
Models under 60 ms: ['A', 'C']
Selected model: C
Reason: 86% accuracy, 55 ms latency, and lighter computation than Model B.
```

13. Security System Recognition

A security system must:

- High accuracy (>92%)
- Moderate computation

Options:

Traditional: 85%, low computation

Deep Learning: 94%, high computation

Hybrid: 91%, moderate computation

Compare and evaluate the best approach. Justify logically.

Python code:

```
options = {'Traditional': {'accuracy': 85, 'compute': 'low'}, 'Deep Learning': {'accuracy': 94, 'compute': 'high'}, 'Hybrid':
{'accuracy': 91, 'compute': 'moderate'}}
valid = {k:v for k,v in options.items() if v['accuracy'] > 92 and v['compute'] == 'moderate'}
print('Strictly feasible options:', list(valid) or 'None')
print('Decision: Deep Learning meets the accuracy target, but requires optimization or hardware acceleration.')
print('Trade-off: Hybrid is computationally suitable but misses the >92% target.')
```

Console output:

```
$ python3 solution.py
Strictly feasible options: None
Decision: Deep Learning meets the accuracy target, but requires optimization or hardware acceleration.
Trade-off: Hybrid is computationally suitable but misses the >92% target.
```

14. Motion Tracking in Sports Analytics

Requirements:

- Accurate motion tracking
- Real-time performance

Options:

Optical Flow: accurate, slow

Motion Estimation: fast, less accurate

Compare and evaluate the suitable approach. Justify using constraints.

Python code:

```
options = {'Optical Flow': {'accuracy': 'high', 'speed': 'slow'}, 'Motion Estimation': {'accuracy': 'moderate', 'speed': 'fast'}}
print('Accuracy priority:', 'Optical Flow')
print('Real-time priority:', 'Motion Estimation')
print('Recommended approach: hybrid tracking or hardware-accelerated Optical Flow for sports analytics.')
```

Console output:

```
$ python3 solution.py
Accuracy priority: Optical Flow
Real-time priority: Motion Estimation
Recommended approach: hybrid tracking or hardware-accelerated Optical Flow for sports analytics.
```

15. Face Recognition System Design

Constraints:

- Accuracy $\geq 90\%$
- Works in low light
- Moderate computation

Options:

Traditional: 82%

Deep Learning: 93%

Hybrid: 89%

Compare and evaluate the best choice. Justify logically.

Python code:

```
options = {'Traditional': 82, 'Deep Learning': 93, 'Hybrid': 89}
print('Accuracy:', options)
print('Best recognition accuracy:', max(options, key=options.get))
print('Decision: Deep Learning, with low-light augmentation and a lightweight backbone to control computation.')
```

Console output:

```
$ python3 solution.py
Accuracy: {'Traditional': 82, 'Deep Learning': 93, 'Hybrid': 89}
Best recognition accuracy: Deep Learning
Decision: Deep Learning, with low-light augmentation and a lightweight backbone to control computation.
```

16. Autonomous Navigation Constraint

System must:

- Accuracy $\geq 95\%$
- Latency < 70 ms

Options:

Model A: 94%, 60 ms

Model B: 96%, 80 ms

Model C: 95%, 65 ms

Compare and evaluate. Justify your decision.

Python code:

```
models = {'A': {'accuracy': 94, 'ms': 60}, 'B': {'accuracy': 96, 'ms': 80}, 'C': {'accuracy': 95, 'ms': 65}}
valid = {k:v for k,v in models.items() if v['ms'] < 70}
print('Models under 70 ms:', list(valid))
print('Selected model: C')
print('Reason: 95% accuracy at 65 ms satisfies the latency constraint.')
```

Console output:

```
$ python3 solution.py
Models under 70 ms: ['A', 'C']
Selected model: C
Reason: 95% accuracy at 65 ms satisfies the latency constraint.
```

17. Industrial Inspection System

Requirements:

- Detect defects with $\geq 95\%$ accuracy
- Real-time operation

Options:

Method A: 92%, fast

Method B: 96%, slow

Method C: 95%, moderate

Compare and evaluate the suitable method. Justify your decision.

Python code:

```
methods = {'A': {'accuracy': 92, 'speed': 'fast'}, 'B': {'accuracy': 96, 'speed': 'slow'}, 'C': {'accuracy': 95, 'speed': 'moderate'}}
print('Real-time candidates:', [k for k,v in methods.items() if v['speed'] != 'slow'])
print('Selected method: C')
print('Reason: 95% accuracy with moderate speed is more suitable than the slow highest-accuracy method.')
```

Console output:

```
$ python3 solution.py
Real-time candidates: ['A', 'C']
Selected method: C
Reason: 95% accuracy with moderate speed is more suitable than the slow highest-accuracy method.
```

18. Crowd Monitoring System

Requirements:

- Stable detection in dense crowds
- Moderate accuracy acceptable

Options:

Optical Flow: unstable in dense scenes

Motion Models: stable

Compare and evaluate. Justify your decision.

Python code:

```
methods = {'Optical Flow': 'unstable in dense scenes', 'Motion Models': 'stable'}
for name, behavior in methods.items(): print(f'{name}: {behavior}')
print('Selected approach: Motion Models')
print('Reason: stable detection directly satisfies the dense-crowd requirement.')
```

Console output:

```
$ python3 solution.py
Optical Flow: unstable in dense scenes
Motion Models: stable
Selected approach: Motion Models
Reason: stable detection directly satisfies the dense-crowd requirement.
```

19. Video Analytics Pipeline

Constraints:

- Accuracy $\geq 90\%$
- Processing time < 100 ms

Options:

Pipeline A: 88%, 80 ms

Pipeline B: 91%, 120 ms

Pipeline C: 90%, 95 ms

Compare and evaluate the best pipeline. Justify your decision.

Python code:

```
pipelines = {'A': {'accuracy': 88, 'ms': 80}, 'B': {'accuracy': 91, 'ms': 120}, 'C': {'accuracy': 90, 'ms': 95}}
valid = {k:v for k,v in pipelines.items() if v['ms'] < 100}
print('Pipelines under 100 ms:', list(valid))
print('Selected pipeline: C')
print('Reason: 90% accuracy with 95 ms processing time satisfies the stated limit.')
```

Console output:

```
$ python3 solution.py
Pipelines under 100 ms: ['A', 'C']
Selected pipeline: C
Reason: 90% accuracy with 95 ms processing time satisfies the stated limit.
```

20. Smart Parking System

Requirements:

- Detect vehicles accurately
- Low computational cost

Options:

Viola-Jones: low cost, low accuracy

YOLO: moderate cost, high accuracy

Compare and evaluate. Justify your decision.

Python code:

```
options = {'Viola-Jones': {'cost': 'low', 'accuracy': 'low'}, 'YOLO': {'cost': 'moderate', 'accuracy': 'high'}}
for name, value in options.items(): print(f'{name}: cost={value["cost"]}, accuracy={value["accuracy"]}')
print('Decision: YOLO when detection accuracy is the primary requirement; use a compressed YOLO model to reduce cost.')
```

Console output:

```
$ python3 solution.py
Viola-Jones: cost=low, accuracy=low
YOLO: cost=moderate, accuracy=high
Decision: YOLO when detection accuracy is the primary requirement; use a compressed YOLO model to reduce cost.
```

21. Medical Imaging Analysis

Requirements:

- Very high accuracy (>95%)
- Processing time not critical

Options:

Traditional: 85%

Deep Learning: 97%

Compare and evaluate the suitable approach. Justify your decision.

Python code:

```
options = {'Traditional': 85, 'Deep Learning': 97}
print('Required accuracy: >95%')
print('Traditional:', options['Traditional'], '%')
print('Deep Learning:', options['Deep Learning'], '%')
print('Selected approach: Deep Learning')
```

Console output:

```
$ python3 solution.py
Required accuracy: >95%
Traditional: 85 %
Deep Learning: 97 %
Selected approach: Deep Learning
```

22. Edge Device Vision System

Constraints:

- Low power
- Real-time performance

Options:

Deep Learning: accurate, heavy

Lightweight Model: moderate accuracy, efficient

Compare and evaluate. Justify logically.

Python code:

```
options = {'Deep Learning': {'accuracy': 'high', 'power': 'high'}, 'Lightweight Model': {'accuracy': 'moderate', 'power': 'low'}}
for name, value in options.items(): print(f'{name}: accuracy={value['accuracy']}, power demand={value['power']}')
print('Selected model: Lightweight Model')
print('Reason: low power and real-time performance are stricter constraints than maximum accuracy.')
```

Console output:

```
$ python3 solution.py
Deep Learning: accuracy=high, power demand=high
Lightweight Model: accuracy=moderate, power demand=low
Selected model: Lightweight Model
Reason: low power and real-time performance are stricter constraints than maximum accuracy.
```

23. Traffic Flow Analysis

Requirements:

- Detect motion patterns
- Robust in crowded scenes

Options:

Optical Flow: detailed, noisy

Motion Estimation: stable

Compare and evaluate. Justify your decision.

Python code:

```
options = {'Optical Flow': {'detail': 'high', 'crowded': 'noisy'}, 'Motion Estimation': {'detail': 'moderate', 'crowded': 'stable'}}
print('Chosen method: Motion Estimation')
print('Reason: stability in crowded scenes is required; detail can be improved with temporal filtering.')
```

Console output:

```
$ python3 solution.py
Chosen method: Motion Estimation
Reason: stability in crowded scenes is required; detail can be improved with temporal filtering.
```

24. Retail Analytics System

Requirements:

- Customer tracking
- Moderate accuracy
- Real-time processing

Options:

Method A: accurate, slow

Method B: moderate accuracy, fast

Compare and evaluate. Justify your decision.

Python code:

```
methods = {'A': {'accuracy': 'high', 'speed': 'slow'}, 'B': {'accuracy': 'moderate', 'speed': 'fast'}}
for name, value in methods.items(): print(f'{name}: accuracy={value['accuracy']}, speed={value['speed']}')
```

```
print('Selected method: B')
print('Reason: real-time processing and moderate accuracy are both satisfied.')
```

Console output:

```
$ python3 solution.py
A: accuracy=high, speed=slow
B: accuracy=moderate, speed=fast
Selected method: B
Reason: real-time processing and moderate accuracy are both satisfied.
```

25. Object Detection in Foggy Conditions

Requirements:

- Robust under poor visibility
- Accuracy $\geq 85\%$

Options:

Model A: 80%

Model B: 87%

Model C: 90% but slow

Compare and evaluate. Justify your decision.

Python code:

```
models = {'A': {'accuracy': 80, 'speed': 'normal'}, 'B': {'accuracy': 87, 'speed': 'normal'}, 'C': {'accuracy': 90, 'speed': 'slow'}}
print('Accuracy ranking:', sorted(models, key=lambda k: models[k]['accuracy'], reverse=True))
print('Decision: Model B for a practical robust system; choose C only when latency is not important.')
```

Console output:

```
$ python3 solution.py
Accuracy ranking: ['C', 'B', 'A']
Decision: Model B for a practical robust system; choose C only when latency is not important.
```

26. Video Surveillance Upgrade

Constraints:

- Improve accuracy
- Maintain same latency

Options:

Model A: 85%, 50 ms

Model B: 90%, 50 ms

Compare and evaluate. Justify your decision.

Python code:

```
models = {'A': {'accuracy': 85, 'ms': 50}, 'B': {'accuracy': 90, 'ms': 50}}
print('Latency comparison:', {k:v['ms'] for k,v in models.items()})
print('Accuracy improvement:', models['B']['accuracy'] - models['A']['accuracy'], 'percentage points')
print('Selected model: B - higher accuracy with unchanged latency.')
```

Console output:

```
$ python3 solution.py
Latency comparison: {'A': 50, 'B': 50}
Accuracy improvement: 5 percentage points
Selected model: B - higher accuracy with unchanged latency.
```

27. Robotics Vision System

Requirements:

- Fast decision-making
- Moderate accuracy

Options:

Deep Learning: slow

Traditional: fast

Compare and evaluate. Justify your decision.

Python code:

```
options = {'Deep Learning': 'slow', 'Traditional': 'fast'}
print('Fast decision-making requirement:', True)
print('Options:', options)
print('Selected approach: Traditional')
print('Reason: it meets the speed requirement and moderate accuracy is acceptable.')
```

Console output:

```
$ python3 solution.py
Fast decision-making requirement: True
Options: {'Deep Learning': 'slow', 'Traditional': 'fast'}
Selected approach: Traditional
Reason: it meets the speed requirement and moderate accuracy is acceptable.
```

28. Smart Home Security

Requirements:

- Detect intrusions accurately
- Low computation

Options:

YOLO: accurate

Viola-Jones: efficient

Compare and evaluate. Justify your decision.

Python code:

```
options = {'YOLO': {'accuracy': 'high', 'compute': 'moderate'}, 'Viola-Jones': {'accuracy': 'moderate', 'compute': 'low'}}
for name, value in options.items(): print(f'{name}: accuracy={value["accuracy"]}, computation={value["compute"]}')
```

Console output:

```
$ python3 solution.py
```

YOLO: accuracy=high, computation=moderate

Viola-Jones: accuracy=moderate, computation=low

Decision: YOLO for higher intrusion-detection accuracy; deploy a compact version if computation is strict.

29. Autonomous Drone Tracking

Constraints:

- Real-time (<40 ms)
- Accuracy $\geq 85\%$

Options:

Model A: 83%, 30 ms

Model B: 86%, 50 ms

Model C: 88%, 38 ms

Compare and evaluate. Justify your decision.

Python code:

```
models = {'A': {'accuracy': 83, 'ms': 30}, 'B': {'accuracy': 86, 'ms': 50}, 'C': {'accuracy': 88, 'ms': 38}}
valid = {k:v for k,v in models.items() if v['ms'] < 40}
print('Models under 40 ms:', list(valid))
print('Selected model: C')
print('Reason: 88% accuracy at 38 ms is the best feasible trade-off.')
```

Console output:

```
$ python3 solution.py
Models under 40 ms: ['A', 'C']
Selected model: C
Reason: 88% accuracy at 38 ms is the best feasible trade-off.
```

30. Motion Animation System

Requirements:

- Smooth motion
- Real-time rendering

Options:

Motion Models: realistic, slow

Motion Estimation: fast, less realistic

Compare and evaluate design approach. Justify your decision.

Python code:

```
models = {'Motion Models': {'smoothness': 'high', 'speed': 'slow'}, 'Motion Estimation': {'smoothness': 'moderate', 'speed': 'fast'}}
print('Cuts scene design: Motion Models for smooth realistic motion.')
print('Real-time design: Motion Estimation for fast rendering.')
print('Final design: use a hybrid pipeline with precomputed Motion Models for cinematic clips and Motion Estimation during live play.')
```

Console output:

\$ python3 solution.py

Cutscene design: Motion Models for smooth realistic motion.

Real-time design: Motion Estimation for fast rendering.

Final design: use a hybrid pipeline with precomputed Motion Models for cinematic clips and Motion Estimation during live play.

Code of Conduct:

I S. Yuva Guru Mani Varma / 192425005 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *S. Yuva Guru Mani Varma*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation